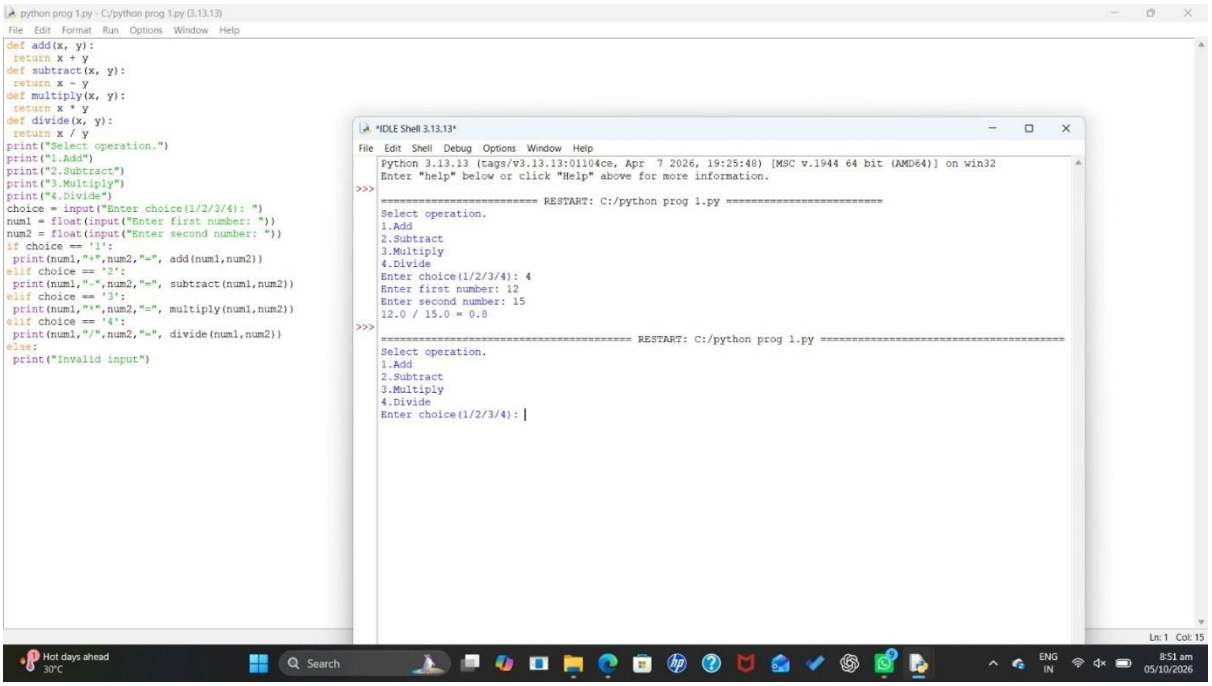
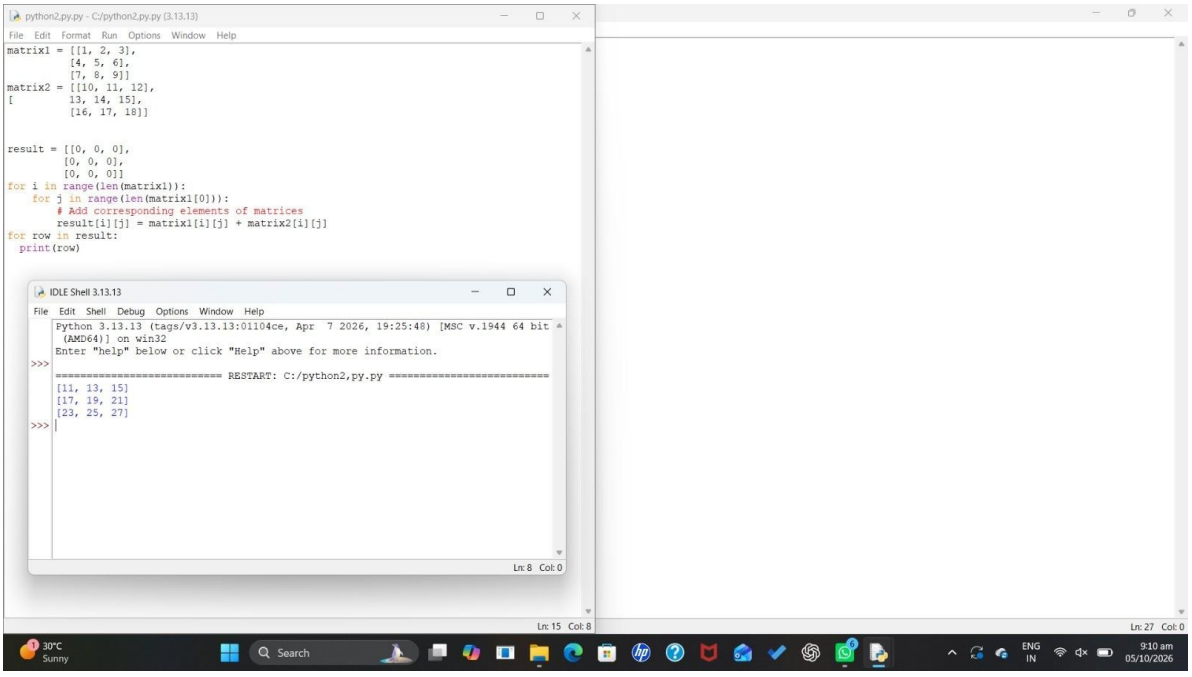
NAME:AAMEENA I(192511284)

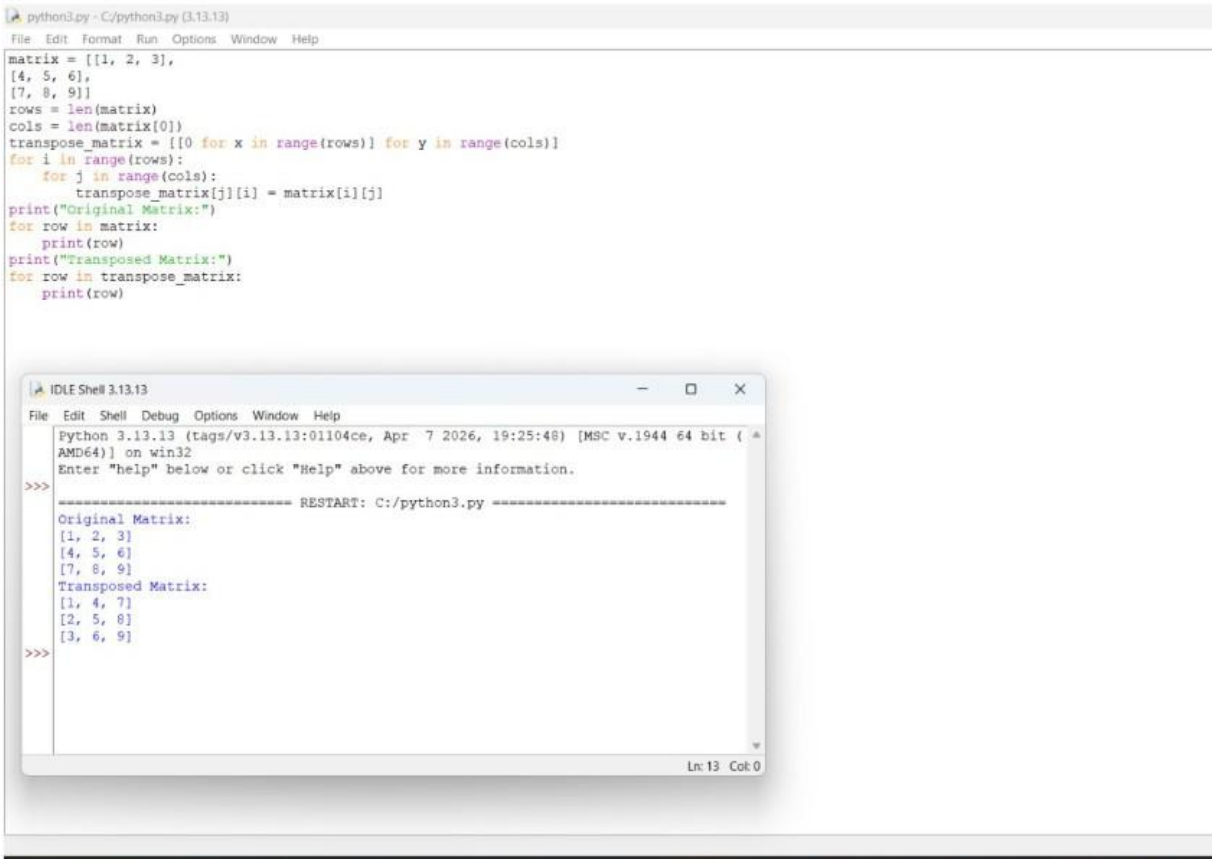
EXPERIMENT 1



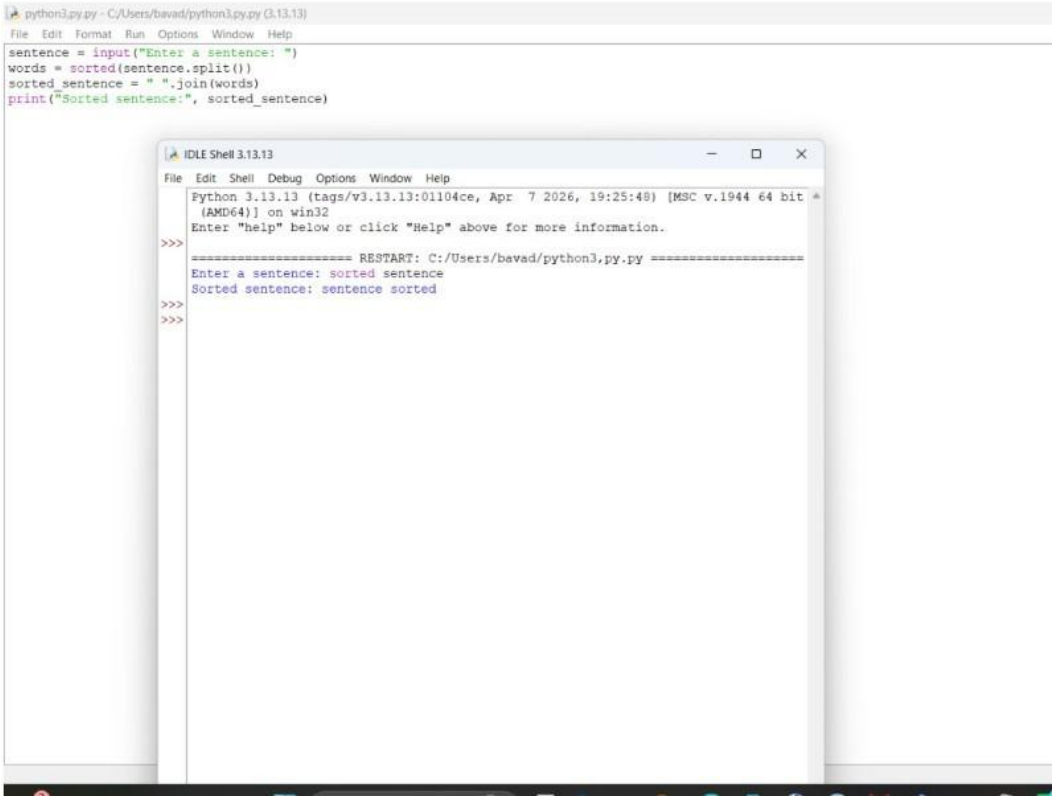
EXPERIMENT 2



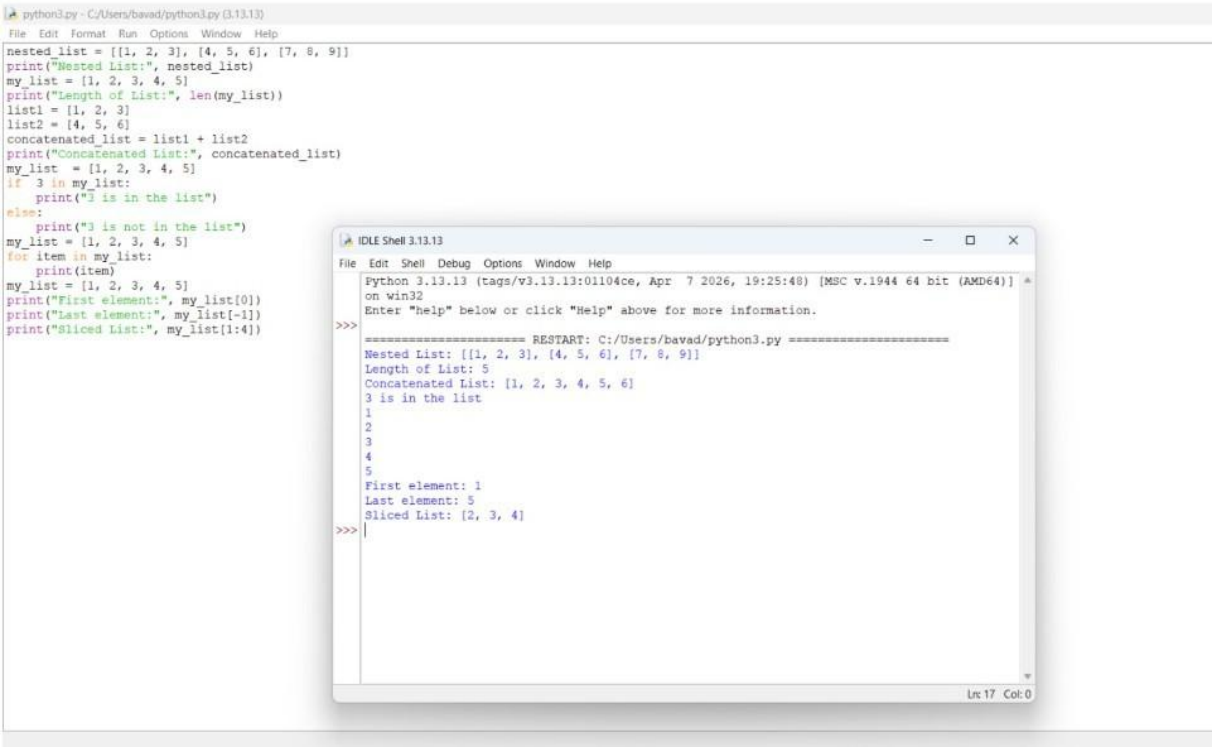
EXPERIMENT 3



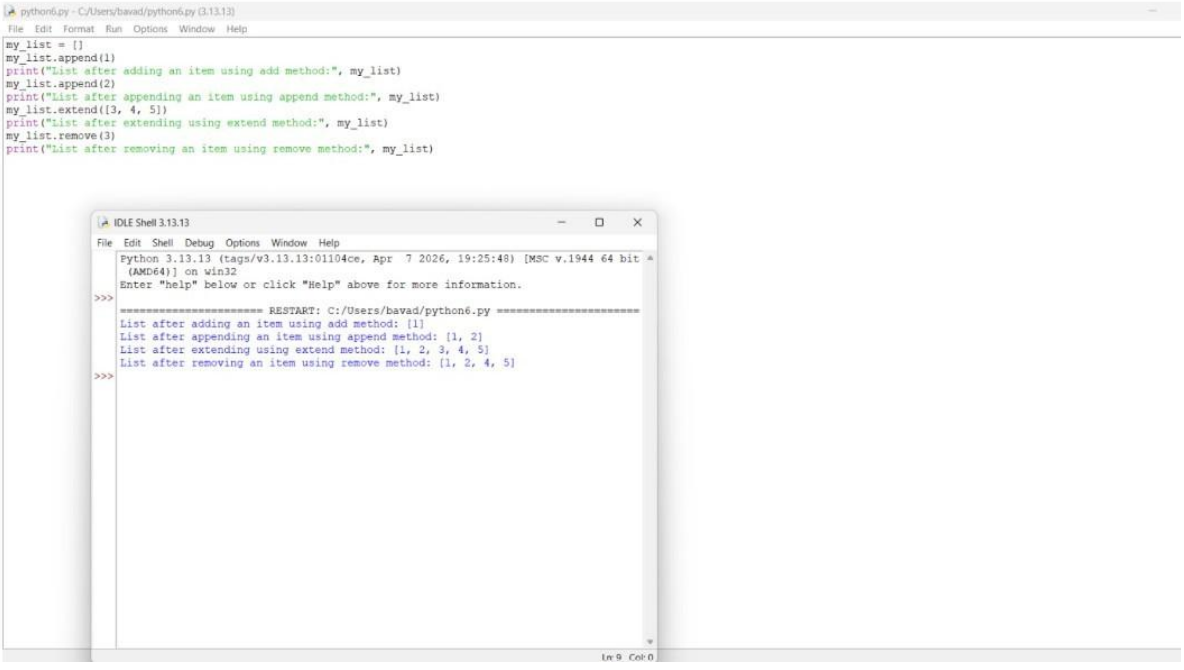
EXPERIMENT 4



EXPERIMENT 5



EXPERIMENT 6



EXPERIMENT 7

```
set1 = {1, 2, 3, 4, 5}
set2 = {4, 5, 6, 7, 8}

print("Set 1:", set1)
print("Set 2:", set2)

intersection_set = set1.intersection(set2)
print("Intersection of set1 and set2:", intersection_set)

union_set = set1.union(set2)
print("Union of set1 and set2:", union_set)

difference_set = set1.difference(set2)
print("Difference between set1 and set2:", difference_set)

symmetric_difference_set = set1.symmetric_difference(set2)
print("Symmetric difference between set1 and set2:", symmetric_difference_set)

subset_check = set1.issubset(set2)
print("Set 1 is subset of set 2:", subset_check)
```

```
Set 1: {1, 2, 3, 4, 5}
Set 2: {4, 5, 6, 7, 8}
Intersection of set1 and set2: {4, 5}
Union of set1 and set2: {1, 2, 3, 4, 5, 6, 7, 8}
Difference between set1 and set2: {1, 2, 3}
Symmetric difference between set1 and set2: {1, 2, 3, 6, 7, 8}
Set 1 is subset of set 2: False
>>>
========= RESTART: C:/Users/bavad/OneDrive/Documents/python 7.py =========
Set 1: {1, 2, 3, 4, 5}
Set 2: {4, 5, 6, 7, 8}
Intersection of set1 and set2: {4, 5}
Union of set1 and set2: {1, 2, 3, 4, 5, 6, 7, 8}
Difference between set1 and set2: {1, 2, 3}
Symmetric difference between set1 and set2: {1, 2, 3, 6, 7, 8}
Set 1 is subset of set 2: False
>>>
```

EXPERIMENT 8

```
import calendar
year = int(input("Enter the year: "))
month = int(input("Enter the month: "))
print(calendar.month(year, month))
```

```
Python 3.13.13 (tags/v3.13.13:01104ce, Apr  7 2026, 19:25:48) [MSC v.1944 64 bit
 (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/bavad/python 8.py ==========
Enter the year: 2008
Enter the month: 1
    January 2008
Mo Tu We Th Fr Sa Su
    1  2  3  4  5  6
 7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30 31

>>>
```

EXPERIMENT 9

```
python 9.py - C:/Users/bavad/python 9.py (3.13.13)
File  Edit  Format  Run  Options  Window  Help
import string
input_string = input("Enter a string: ")
no_punctuations = input_string.translate(str.maketrans("", "", string.punctuation))
print("String with no punctuations:", no_punctuations)
```

```
IDLE Shell 3.13.13
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.13 (tags/v3.13.13:01104ce, Apr  7 2026, 19:25:48) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTA
RT: C:/Users/bavad/python 9.py ==========
==========
Enter a string: Hello
String with no punctuations: Hello
>>>
```

EXPERIMENT 10

```
python 10.py - C:/Users/bavad/python 10.py (3.13.13)
File  Edit  Format  Run  Options  Window  Help
from queue import PriorityQueue

# Goal state
goal_state = (1, 2, 3,
              4, 5, 6,
              7, 8, 0)


# Manhattan Distance Heuristic
def heuristic(state):
    distance = 0

    for i in range(3):
        for j in range(3):
            tile = state[i * 3 + j]

            if tile != 0:
                x = (tile - 1) // 3
                y = (tile - 1) % 3

                distance += abs(x - i) + abs(y - j)

    return distance


# Generate successor states
def successors(state):
    successors_list = []

    # Position of blank (0)
    i = state.index(0)

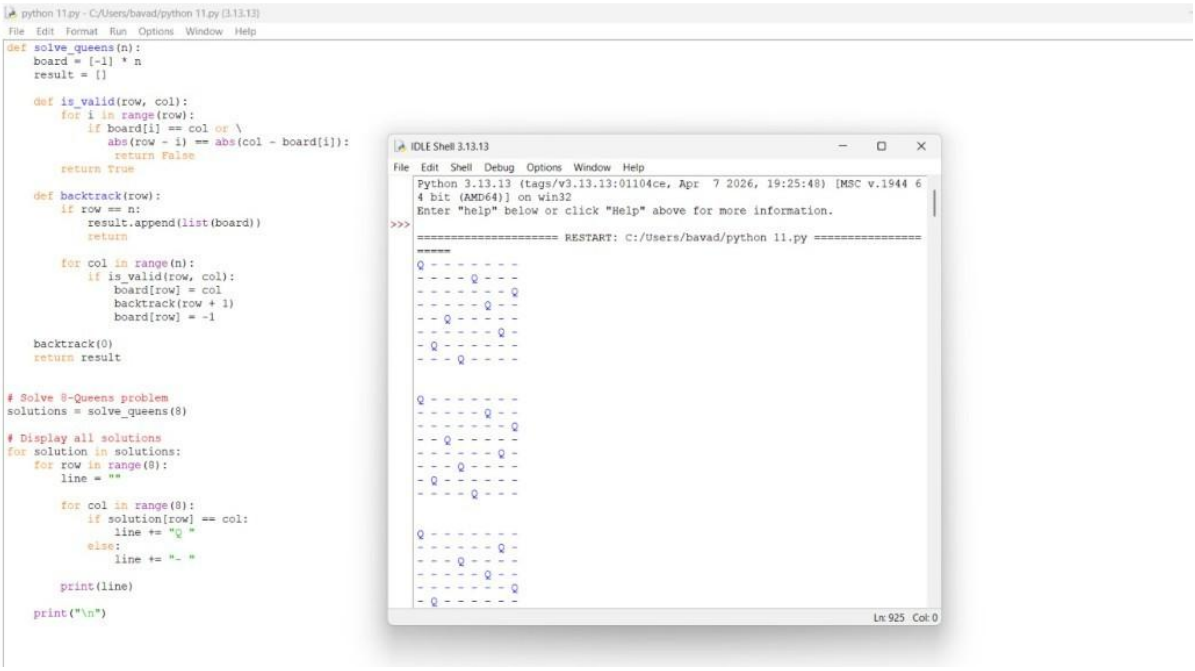
    # Slide tile to the left
    if i % 3 != 0:
        new_state = list(state)
        new_state[i], new_state[i - 1] = new_state[i - 1], new_state[i]
        successors_list.append(tuple(new_state))

    # Slide tile to the right
    if i % 3 != 2:
        new_state = list(state)
        new_state[i], new_state[i + 1] = new_state[i + 1], new_state[i]
        successors_list.append(tuple(new_state))

    # Slide tile up
    if i // 3 != 0:
        new_state = list(state)
```

```
IDLE Shell 3.13.13
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.13 (tags/v3.13.13:01104ce, Apr  7 2026, 19:25:48) [MSC v.1944
64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/bavad/python 10.py ==========
======
The puzzle is unsolvable.
>>>
Ln: 6  Col: 0
```

EXPERIMENT 11



```
def solve_queens(n):
    board = [-1] * n
    result = []

    def is_valid(row, col):
        for i in range(row):
            if board[i] == col or \
               abs(row - i) == abs(col - board[i]):
                return False
        return True

    def backtrack(row):
        if row == n:
            result.append(list(board))
            return

        for col in range(n):
            if is_valid(row, col):
                board[row] = col
                backtrack(row + 1)
                board[row] = -1

    backtrack(0)
    return result


# Solve 8-Queens problem
solutions = solve_queens(8)

# Display all solutions
for solution in solutions:
    for row in range(8):
        line = ""

        for col in range(8):
            if solution[row] == col:
                line += "Q "
            else:
                line += "- "

        print(line)

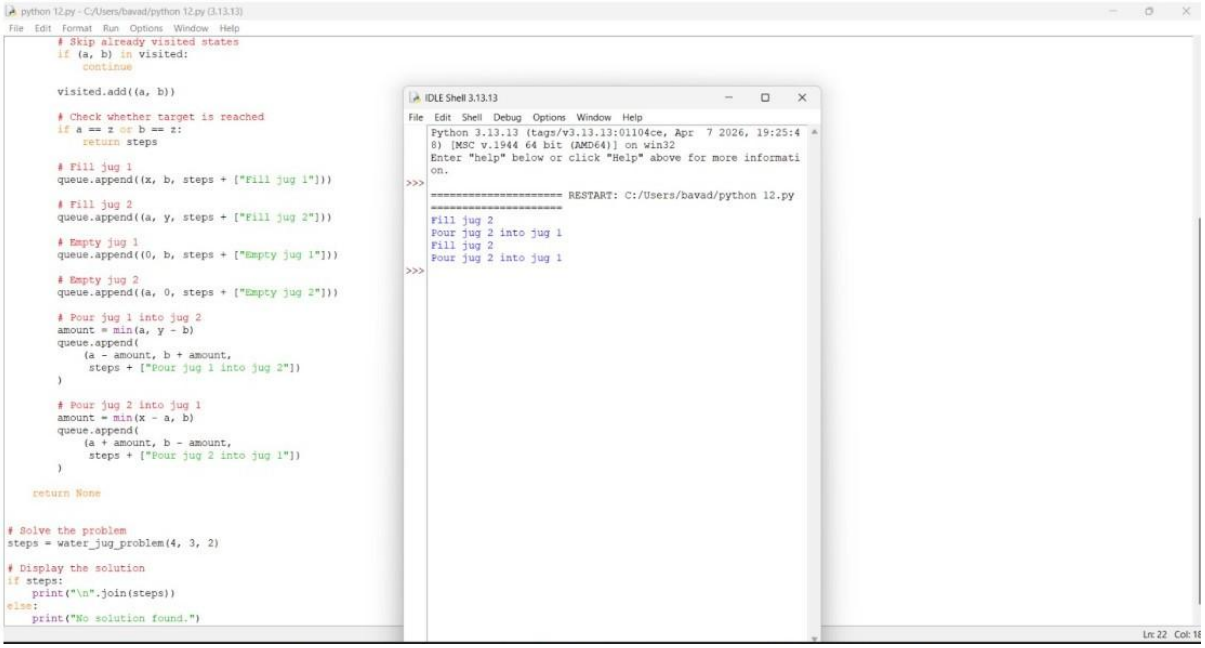
    print("\n")
```

```
Python 3.13.13 (tags/v3.13.13:01104ce, Apr  7 2026, 19:25:48) [MSC v.1944 6
4 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/bavad/python 11.py ==========
=====
Q - - - - - - -
- - - - Q - - -
- - - - - - - Q
- - - - - Q - -
- - Q - - - - -
- - - - - - Q -
- Q - - - - - -
- - - Q - - - -


Q - - - - - - -
- - - - - Q - -
- - - - - - - Q
- - Q - - - - -
- - - - - - Q -
- - - Q - - - -
- Q - - - - - -
- - - - Q - - -


Q - - - - - - -
- - - - - - Q -
- - - Q - - - -
- - - - - Q - -
- - - - - - - Q
- Q - - - - - -
```

EXPERIMENT 12



```
    # Skip already visited states
    if (a, b) in visited:
        continue

    visited.add((a, b))

    # Check whether target is reached
    if a == z or b == z:
        return steps

    # Fill jug 1
    queue.append((x, b, steps + ["Fill jug 1"]))

    # Fill jug 2
    queue.append((a, y, steps + ["Fill jug 2"]))

    # Empty jug 1
    queue.append((0, b, steps + ["Empty jug 1"]))

    # Empty jug 2
    queue.append((a, 0, steps + ["Empty jug 2"]))

    # Pour jug 1 into jug 2
    amount = min(a, y - b)
    queue.append(
        (a - amount, b + amount,
         steps + ["Pour jug 1 into jug 2"])
    )

    # Pour jug 2 into jug 1
    amount = min(x - a, b)
    queue.append(
        (a + amount, b - amount,
         steps + ["Pour jug 2 into jug 1"])
    )

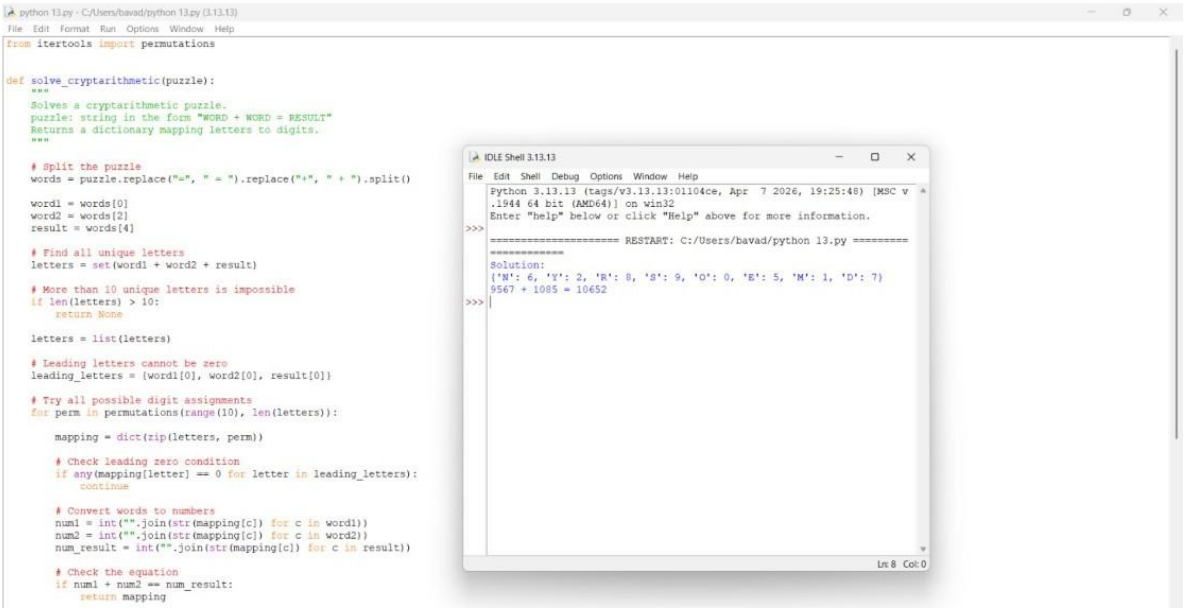
return None


# Solve the problem
steps = water_jug_problem(4, 3, 2)

# Display the solution
if steps:
    print("\n".join(steps))
else:
    print("No solution found.")
```

```
Python 3.13.13 (tags/v3.13.13:01104ce, Apr  7 2026, 19:25:4
8) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more informati
on.
>>>
========== RESTART: C:/Users/bavad/python 12.py
==========
Fill jug 2
Pour jug 2 into jug 1
Fill jug 2
Pour jug 2 into jug 1
>>>
```

EXPERIMENT 13



```
from itertools import permutations

def solve_cryptarithmetic(puzzle):
    """
    Solves a cryptarithmetic puzzle.
    puzzle: string in the form "WORD + WORD = RESULT"
    Returns a dictionary mapping letters to digits.
    """

    # Split the puzzle
    words = puzzle.replace("=", " = ").replace("+", " + ").split()

    word1 = words[0]
    word2 = words[2]
    result = words[4]

    # Find all unique letters
    letters = set(word1 + word2 + result)

    # More than 10 unique letters is impossible
    if len(letters) > 10:
        return None

    letters = list(letters)

    # Leading letters cannot be zero
    leading_letters = {word1[0], word2[0], result[0]}

    # Try all possible digit assignments
    for perm in permutations(range(10), len(letters)):

        mapping = dict(zip(letters, perm))

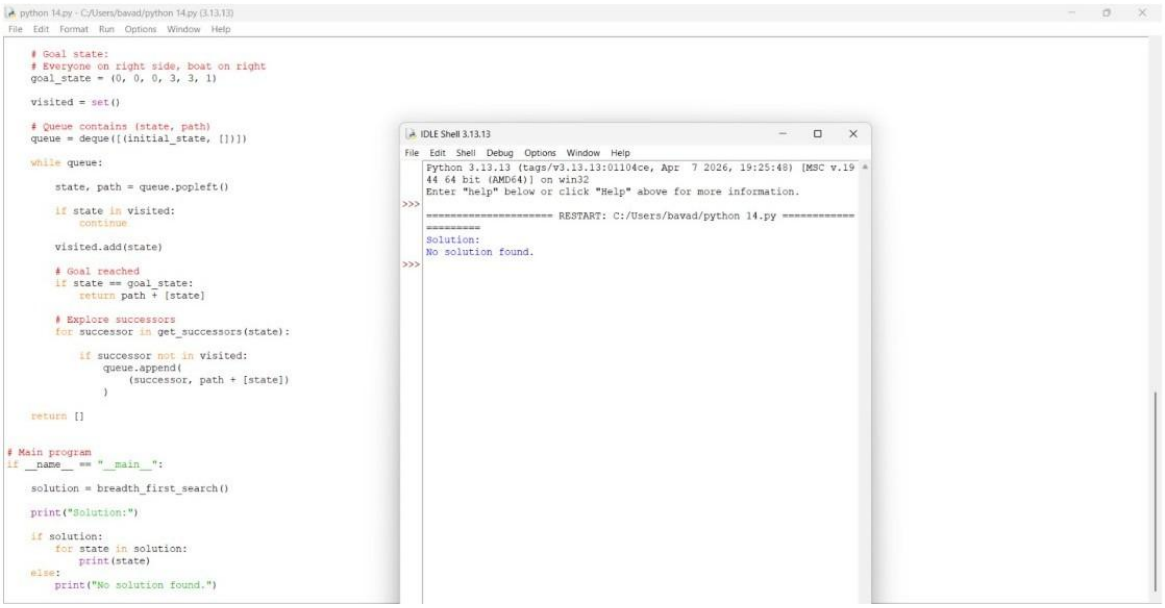
        # Check leading zero condition
        if any(mapping[letter] == 0 for letter in leading_letters):
            continue

        # Convert words to numbers
        num1 = int("".join(str(mapping[c]) for c in word1))
        num2 = int("".join(str(mapping[c]) for c in word2))
        num_result = int("".join(str(mapping[c]) for c in result))

        # Check the equation
        if num1 + num2 == num_result:
            return mapping
```

```
IDLE Shell 3.13.13
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.13 (tags/v3.13.13:01104ce, Apr  7 2026, 19:25:48) [MSC v
.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/bavad/python 13.py ==========
==========
Solution:
{'N': 6, 'Y': 2, 'R': 8, 'S': 9, 'O': 0, 'E': 5, 'M': 1, 'D': 7}
9567 + 1085 = 10652
>>>
Ln: 8  Col: 0
```

EXPERIMENT 14



```
    # Goal state:
    # Everyone on right side, boat on right
    goal_state = (0, 0, 0, 3, 3, 1)

    visited = set()

    # Queue contains (state, path)
    queue = deque([(initial_state, [])])

    while queue:

        state, path = queue.popleft()

        if state in visited:
            continue

        visited.add(state)

        # Goal reached
        if state == goal_state:
            return path + [state]

        # Explore successors
        for successor in get_successors(state):

            if successor not in visited:
                queue.append(
                    (successor, path + [state])
                )

    return []


# Main program
if __name__ == "__main__":

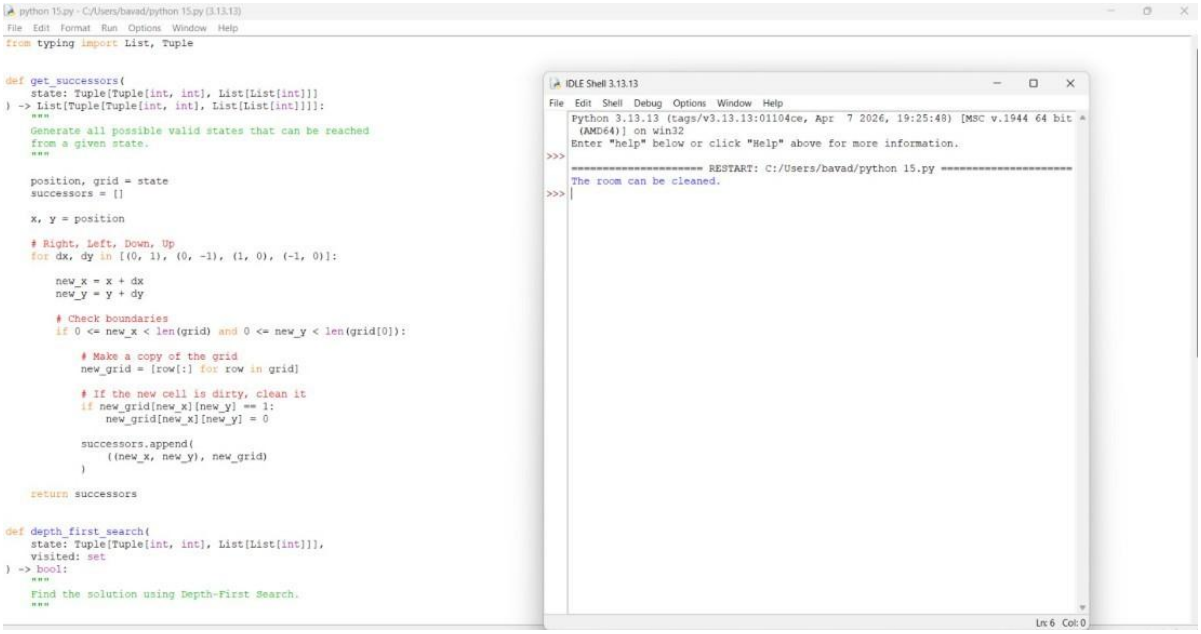
    solution = breadth_first_search()

    print("Solution:")

    if solution:
        for state in solution:
            print(state)
    else:
        print("No solution found.")
```

```
IDLE Shell 3.13.13
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.13 (tags/v3.13.13:01104ce, Apr  7 2026, 19:25:48) [MSC v.19
44 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/bavad/python 14.py ==========
==========
Solution:
No solution found.
>>>
```

EXPERIMENT 15



```
python 15.py - C:/Users/bavad/python 15.py (3.13.13)
File  Edit  Format  Run  Options  Window  Help
from typing import List, Tuple

def get_successors(
    state: Tuple[Tuple[int, int], List[List[int]]]
) -> List[Tuple[Tuple[int, int], List[List[int]]]]:
    """
    Generate all possible valid states that can be reached
    from a given state.
    """

    position, grid = state
    successors = []

    x, y = position

    # Right, Left, Down, Up
    for dx, dy in [(0, 1), (0, -1), (1, 0), (-1, 0)]:

        new_x = x + dx
        new_y = y + dy

        # Check boundaries
        if 0 <= new_x < len(grid) and 0 <= new_y < len(grid[0]):

            # Make a copy of the grid
            new_grid = [row[:] for row in grid]

            # If the new cell is dirty, clean it
            if new_grid[new_x][new_y] == 1:
                new_grid[new_x][new_y] = 0

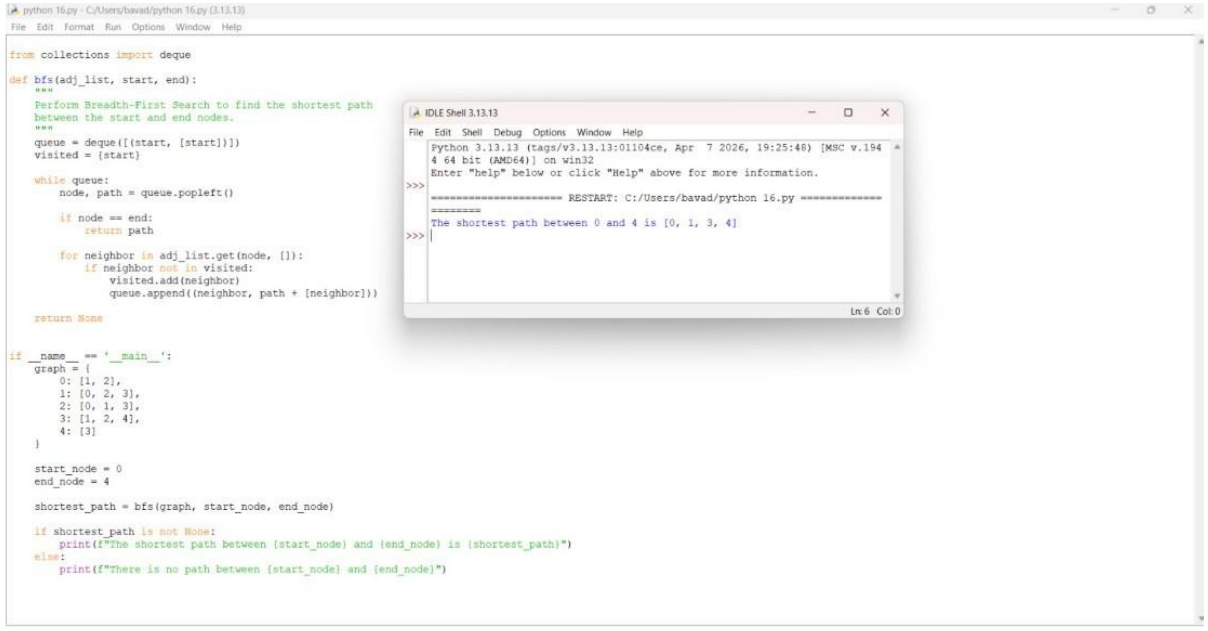
            successors.append(
                ((new_x, new_y), new_grid)
            )

    return successors

def depth_first_search(
    state: Tuple[Tuple[int, int], List[List[int]]],
    visited: set
) -> bool:
    """
    Find the solution using Depth-First Search.
    """
```

```
IDLE Shell 3.13.13
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.13 (tags/v3.13.13:01104ce, Apr  7 2026, 19:25:48) [MSC v.1944 64 bit
 (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/bavad/python 15.py ==========
The room can be cleaned.
>>>
Ln: 6  Col: 0
```

EXPERIMENT 16



```
python 16.py - C:/Users/bavad/python 16.py (3.13.13)
File  Edit  Format  Run  Options  Window  Help
from collections import deque

def bfs(adj_list, start, end):
    """
    Perform Breadth-First Search to find the shortest path
    between the start and end nodes.
    """
    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        node, path = queue.popleft()

        if node == end:
            return path

        for neighbor in adj_list.get(node, []):
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append((neighbor, path + [neighbor]))

    return None


if __name__ == '__main__':
    graph = {
        0: [1, 2],
        1: [0, 2, 3],
        2: [0, 1, 3],
        3: [1, 2, 4],
        4: [3]
    }

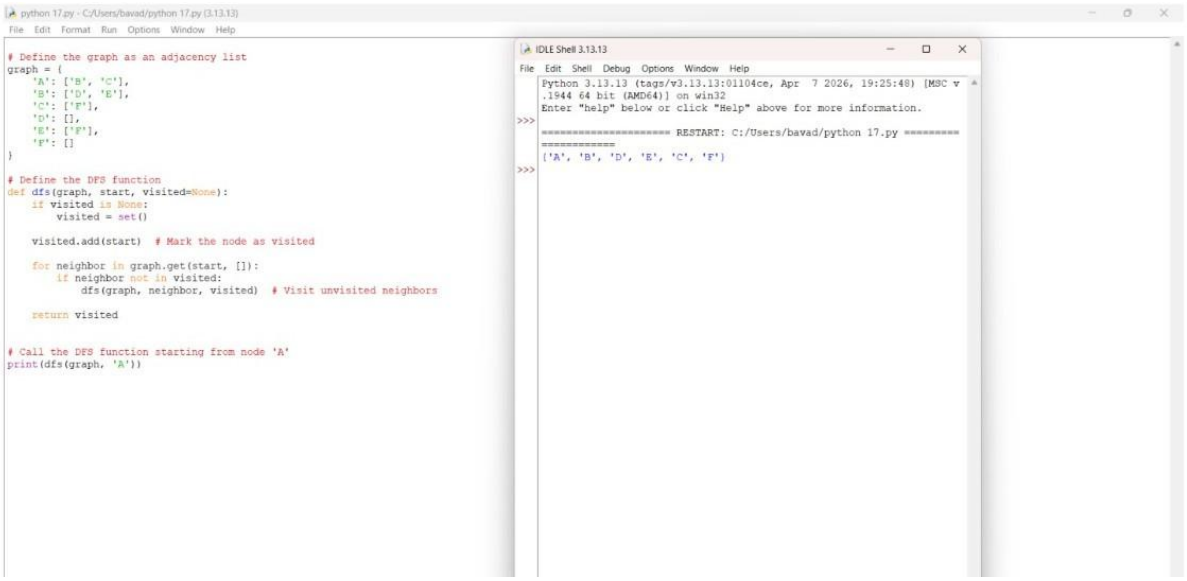
    start_node = 0
    end_node = 4

    shortest_path = bfs(graph, start_node, end_node)

    if shortest_path is not None:
        print(f"The shortest path between {start_node} and {end_node} is {shortest_path}")
    else:
        print(f"There is no path between {start_node} and {end_node}")
```

```
IDLE Shell 3.13.13
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.13 (tags/v3.13.13:01104ce, Apr  7 2026, 19:25:48) [MSC v.194
4 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/bavad/python 16.py ==========
========
The shortest path between 0 and 4 is [0, 1, 3, 4]
>>>
Ln: 6  Col: 0
```

EXPERIMENT 17



```
# Define the graph as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F'],
    'D': [],
    'E': ['F'],
    'F': []
}

# Define the DFS function
def dfs(graph, start, visited=None):
    if visited is None:
        visited = set()

    visited.add(start)  # Mark the node as visited

    for neighbor in graph.get(start, []):
        if neighbor not in visited:
            dfs(graph, neighbor, visited)  # Visit unvisited neighbors

    return visited


# Call the DFS function starting from node 'A'
print(dfs(graph, 'A'))
```

```
Python 3.13.13 (tags/v3.13.13:01104ce, Apr  7 2026, 19:25:48) [MSC v
.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/bavad/python 17.py ==========
{'A', 'B', 'D', 'E', 'C', 'F'}
>>>
```

EXPERIMENT 18

```
from itertools import permutations

def tsp(graph, start):
    n = len(graph)
    cities = list(range(n))
    cities.remove(start)

    min_cost = float('inf')
    best_path = []

    for path in permutations(cities):
        current_cost = 0
        k = start

        for city in path:
            current_cost += graph[k][city]
            k = city

        current_cost += graph[k][start]

        if current_cost < min_cost:
            min_cost = current_cost
            best_path = [start] + list(path) + [start]

    return min_cost, best_path


graph = [
    [0, 10, 15, 20],
    [10, 0, 35, 25],
    [15, 35, 0, 30],
    [20, 25, 30, 0]
]

cost, path = tsp(graph, 0)

print("Minimum cost:", cost)
print("Optimal path:", " -> ".join(map(str, path)))
```

```
Python 3.13.13 (tags/v3.13.13:01104ce, Apr  7 2026, 19:25:48) [MSC v
.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/bavad/PYTHON 18.py ==========
Minimum cost: 80
Optimal path: 0 -> 1 -> 3 -> 2 -> 0
>>>
```

EXPERIMENT 19



```
PYTHON19.py - C:/Users/bavad/PYTHON19.py (3.13.13)
File  Edit  Format  Run  Options  Window  Help
        )

        if current == goal:
            path = []
            while current is not None:
                path.append(current)
                current = parent[current]

            path.reverse()
            print("Path:", path)
            print("Total cost:", g_cost[goal])
            return

        open_list.remove(current)

        for neighbor, cost in graph[current].items():
            new_cost = g_cost[current] + cost

            if neighbor not in g_cost or new_cost < g_cost[neighbor]:
                g_cost[neighbor] = new_cost
                parent[neighbor] = current

                if neighbor not in open_list:
                    open_list.append(neighbor)

    print("No path found")


graph = {
    'A': {'B': 1, 'C': 4},
    'B': {'D': 2, 'E': 5},
    'C': {'E': 1},
    'D': {'G': 5},
    'E': {'G': 3},
    'G': {}
}

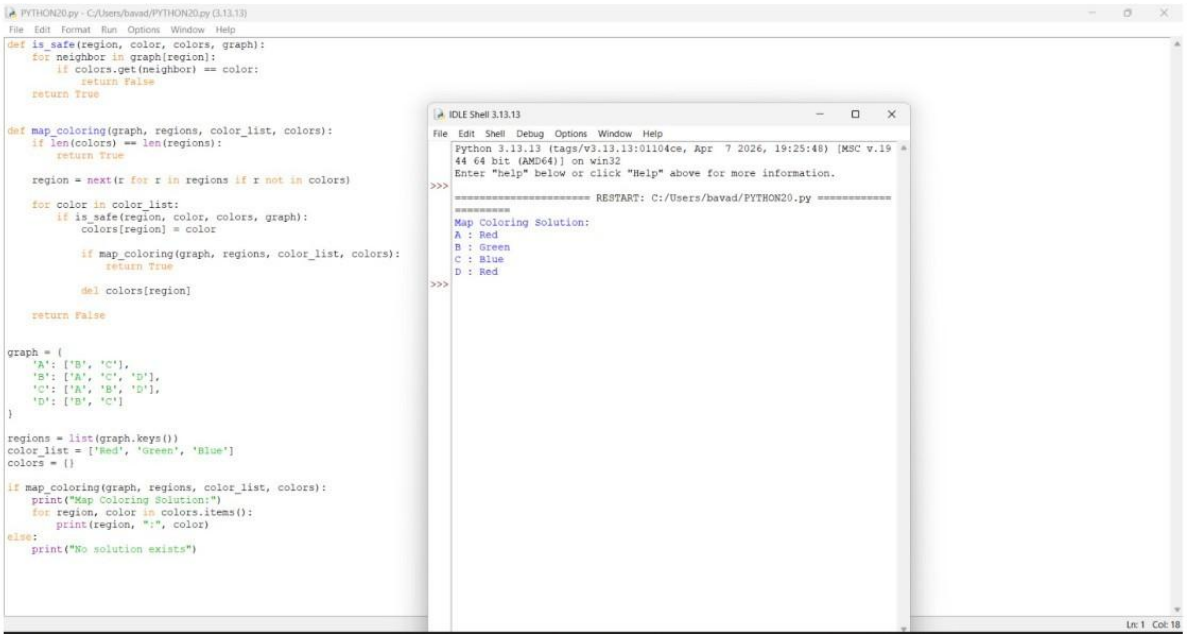
heuristic = {
    'A': 7,
    'B': 6,
    'C': 4,
    'D': 4,
    'E': 2,
    'G': 0
}

astar(graph, heuristic, 'A', 'G')
```



```
IDLE Shell 3.13.13
File  Edit  Shell  Debug  Options  Window  Help
Python 3.13.13 (tags/v3.13.13:01104ce, Apr  7 2026, 19:25:48) [MSC v
.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
================== RESTART: C:/Users/bavad/PYTHON19.py =========
Path: ['A', 'B', 'D', 'G']
Total cost: 8
>>>
```

EXPERIMENT 20


PYTHON20.py - C:/Users/bavad/PYTHON20.py (3.13.13)
File Edit Format Run Options Window Help
def is_safe(region, color, colors, graph):
for neighbor in graph[region]:
if colors.get(neighbor) == color:
return False
return True
def map_coloring(graph, regions, color_list, colors):
if len(colors) == len(regions):
return True
region = next(r for r in regions if r not in colors)
for color in color_list:
if is_safe(region, color, colors, graph):
colors[region] = color
if map_coloring(graph, regions, color_list, colors):
return True
del colors[region]
return False
graph = {
'A': ['B', 'C'],
'B': ['A', 'C', 'D'],
'C': ['A', 'B', 'D'],
'D': ['B', 'C']
}
regions = list(graph.keys())
color_list = ['Red', 'Green', 'Blue']
colors = {}
if map_coloring(graph, regions, color_list, colors):
print("Map Coloring Solution:")
for region, color in colors.items():
print(region, ":", color)
else:
print("No solution exists")
Ln: 1 Col: 18
IDLE Shell 3.13.13
File Edit Shell Debug Options Window Help
Python 3.13.13 (tags/v3.13.13:01104ce, Apr 7 2026, 19:25:48) [MSC v.19
44 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/bavad/PYTHON20.py ==========
Map Coloring Solution:
A : Red
B : Green
C : Blue
D : Red
>>>